DelayLens
Viva Preparation Guide

What we built, the role of Hadoop, every MapReduce job explained, and the questions you will be asked

Project: DelayLens: Agentic Big Data Analytics for Delivery-Delay Intelligence

Course: Big Data Analytics, Continuous Assessment 3 (Case Study) | Symbiosis Institute of Technology, AI & ML

Team: Arav Saxena (PRN 23070126018) & Mahi Laddha (PRN 230701260169)

Viva window: 12 to 15 October 2026 | **Submission:** 15 October 2026 (Moodle)

> **How to use this guide.** Read Part 1 and 2 first (the story). Parts 3 to 6 are the Hadoop knowledge you must be able to explain with the diagrams. Part 7 covers the cluster setup and problems you solved, Part 8 the rest of the system. Part 9 is the question bank with model answers (say the answer in your own words, do not memorise it), Part 10 the live demo and Part 11 the numbers cheat-sheet. Every number in this guide is a real measured value from our runs.

1 The 60-second story
2 What we did, step by step
3 Hadoop from zero (HDFS, YARN, MapReduce)
4 The role of Hadoop in DelayLens
5 All 33 jobs explained
6 Performance and correctness
7 Cluster setup and problems solved
8 The rest of the system in brief
9 Viva question bank (60+ questions)
10 Live demo script
11 Numbers cheat-sheet

1. The 60-second story

> **Say this first when asked "tell us about your project":**
> "DataCo, a global retailer, delivered **54.8% of its 65,752 orders late**. We stored its order and web-log data on **HDFS** and wrote **33 MapReduce jobs on YARN** that compute delivery KPIs by region, shipping mode, product and month, and re-score orders under different delivery promises. The results go into **MongoDB**, and a **LangGraph AI copilot** and a **Next.js dashboard** read them. We found that delays are caused by **shipping-mode promises, not geography**: First Class promises 1 day but always takes 2 (95.3% late). Better promises would cut lateness to **30.3%**, or **18.9%** with a 5-day Standard promise."

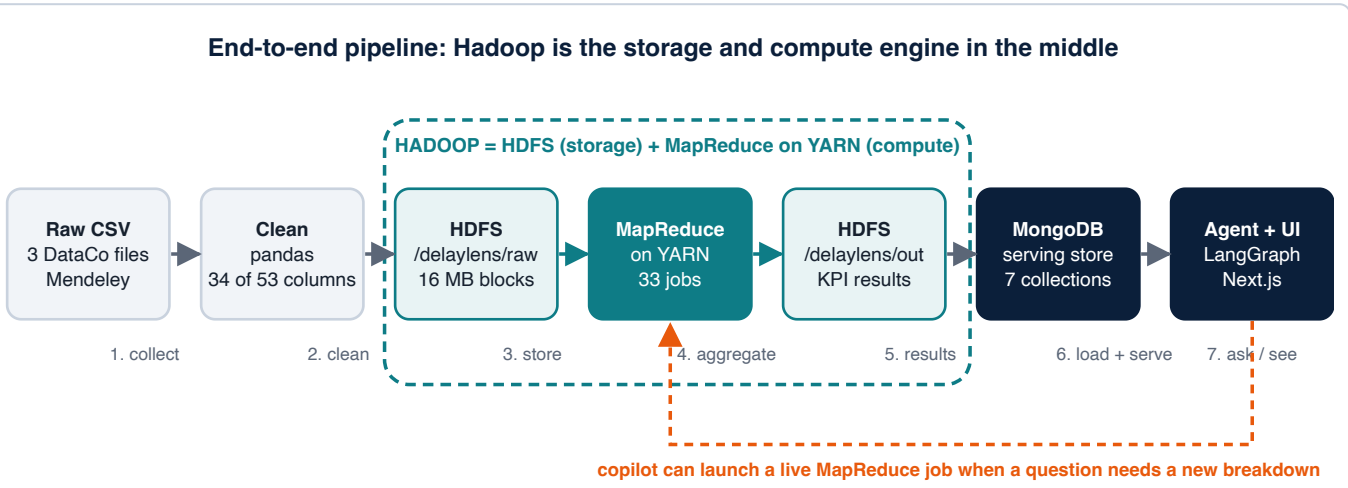


Figure 1. The whole system. The dashed zone is Hadoop: it stores the data (HDFS) and does all heavy aggregation (MapReduce on YARN). MongoDB only serves the finished results.

The problem, the idea and the answer

Question	Our answer
Problem	54.8% of orders (36,048 of 65,752) arrive after the promised date. Managers cannot hand-write a query for every question as data and users grow.
Why Big Data tools	180,519 order lines + 469,977 log lines, many group-by questions over the same data, plus a need to scale. Hadoop gives parallel batch aggregation; MongoDB gives fast ad-hoc answers; an agent gives natural-language access.
Main finding	The cause is the **promise attached to each shipping mode**: First Class 95.3% late, Second Class 76.7%, Same Day 46.2%, Standard 38.1%. Regions (49.5% to 57.6%), markets, categories, segments and months barely matter.
What would fix it	Hadoop what-if jobs: First Class 2 days, Second Class 4 days, Same Day cut-off 12:00 gives 30.3% (16,145 fewer late orders); plus Standard 5 days gives 18.9% (23,603 fewer).
Who benefits now	11,674 open orders worth $6.58M sit in segments that are late at least 75% of the time: customer service can contact them first.

2. What we did, step by step

#	Step	What happened	Key numbers
1	Choose problem + dataset	DataCo Smart Supply Chain (Mendeley Data): orders, delivery status, promised vs actual days, plus web access logs. Chosen because it has a clear business problem (late delivery) and two data types (orders, clicks).	180,519 lines, 53 fields; 469,977 log lines
2	Install Hadoop	Hadoop 3.5.0, OpenJDK 17, pseudo-distributed on one Mac: NameNode, DataNode, ResourceManager, NodeManager, JobHistoryServer. Started without SSH. NameNode on port 9820.	16 MB blocks, replication 1, YARN 8 GB / 8 vcores
3	Profile + clean (pandas)	Kept 34 of 53 columns, translated Spanish country/state names, ISO dates, derived delay_days / is_late / is_canceled / order month, weekday, hour; removed duplicate log lines; exported tab-separated files. Verified the late label.	3,249 duplicate log lines removed; label 100% consistent
4	Store on HDFS	`hdfs dfs -put` of the two cleaned files into /delaylens/raw.	44.6 MB + 48.3 MB, 3 blocks each, 3.2 s
5	MapReduce: Job A	ETL job that rolls order lines up to one record per order.	180,519 lines to 65,752 orders
6	MapReduce: KPI jobs	16 order-level and 4 item-level KPI jobs with one generic mapper/reducer; 6 clickstream jobs; 5 promise what-if jobs; 1 native Java job.	33 jobs, 533.5 s wall time
7	Benchmark	Re-ran the region KPI job on 0.25x to 16x data with Streaming, Java and pandas, with and without a combiner.	11 MB to 723 MB; shuffle 2,600x smaller
8	Load MongoDB	Orders as nested documents, all KPI outputs, scenarios, click counts, job counters; indexes built. Queries written and timed. Hadoop-vs-MongoDB consistency check.	607 KPI rows; 244.6 MB data, 38.4 MB on disk
9	Agentic copilot	LangGraph StateGraph: planner (LLM function calling) to tools to analyst, plus an offline fallback. 9 tools including one that launches a live MapReduce job.	8 of 8 test questions correct
10	Dashboard	Next.js 16 with 8 pages and the copilot as a side panel.	8 pages, Recharts
11	Report + presentation	10-section report in the required format with SIT cover page; 10-slide deck.	32-page PDF

Scope decisions you should be ready to justify. (1) **No machine learning**: the analysis shows lateness follows fixed promise rules, so a what-if simulation explains it more directly than a classifier. (2) **Single-node Hadoop**: real Hadoop software and APIs, one machine; the same jobs run unchanged on a multi-node cluster. (3) **MapReduce, not Spark**: the course platform; writing mappers and reducers shows how partition, shuffle and combine work.

3. Hadoop from zero, in our project's words

Hadoop has three layers. Each is shown below with the real values from our cluster.

3.1 HDFS: where the data lives

- **Blocks.** HDFS cuts every file into fixed-size blocks and spreads them over DataNodes. Default size is 128 MB; we set **16 MB** so our 44.6 MB file becomes **3 blocks** (16 + 16 + 12.6 MB), which gives 3 parallel map tasks.
- **NameNode** keeps only metadata (file to blocks to locations) in memory. **DataNode** stores the blocks and sends heartbeats and block reports.
- **Replication** keeps copies of each block on different nodes. We use **1** because we have one DataNode; production uses 3.
- **Write once, read many.** Files are loaded with `hdfs dfs -put` and then only read by jobs; results are written as new directories.

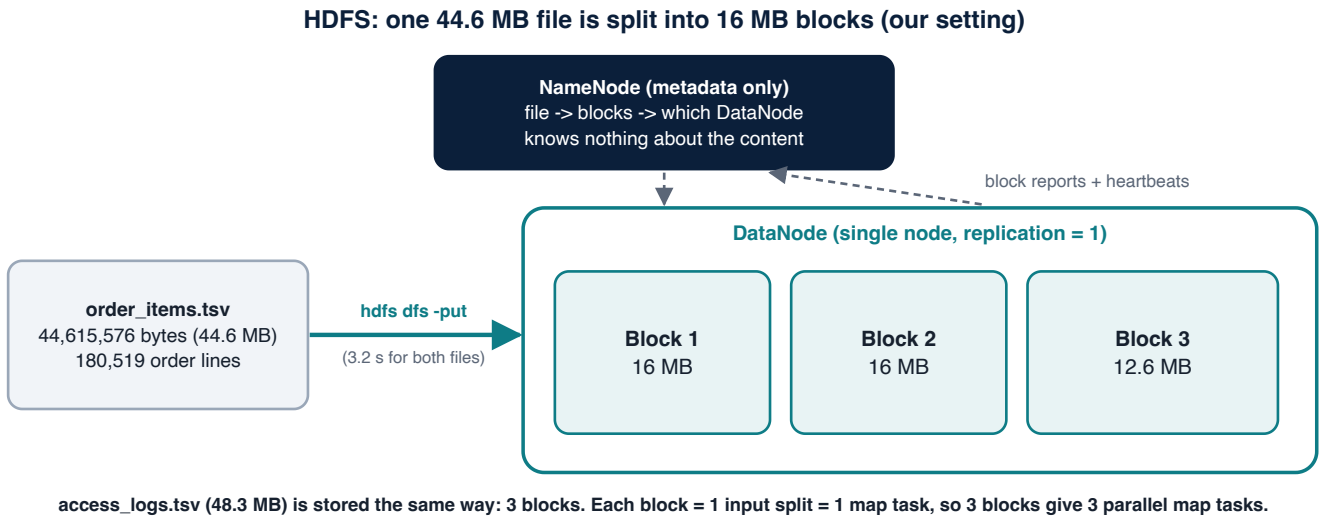


Figure 2. Our order file on HDFS. Each block becomes one input split and one map task.

3.2 YARN: who runs the tasks

- **ResourceManager** owns the cluster resources (we gave it 8 GB and 8 vcores) and schedules containers.
- **NodeManager** runs containers on its machine. A **container** is a slice of CPU and memory that holds one task.
- **ApplicationMaster** is started once per job; it asks the ResourceManager for containers for the map and reduce tasks and watches them.
- **JobHistoryServer** keeps counters and logs of finished jobs (port 19888); we read job counters from it.

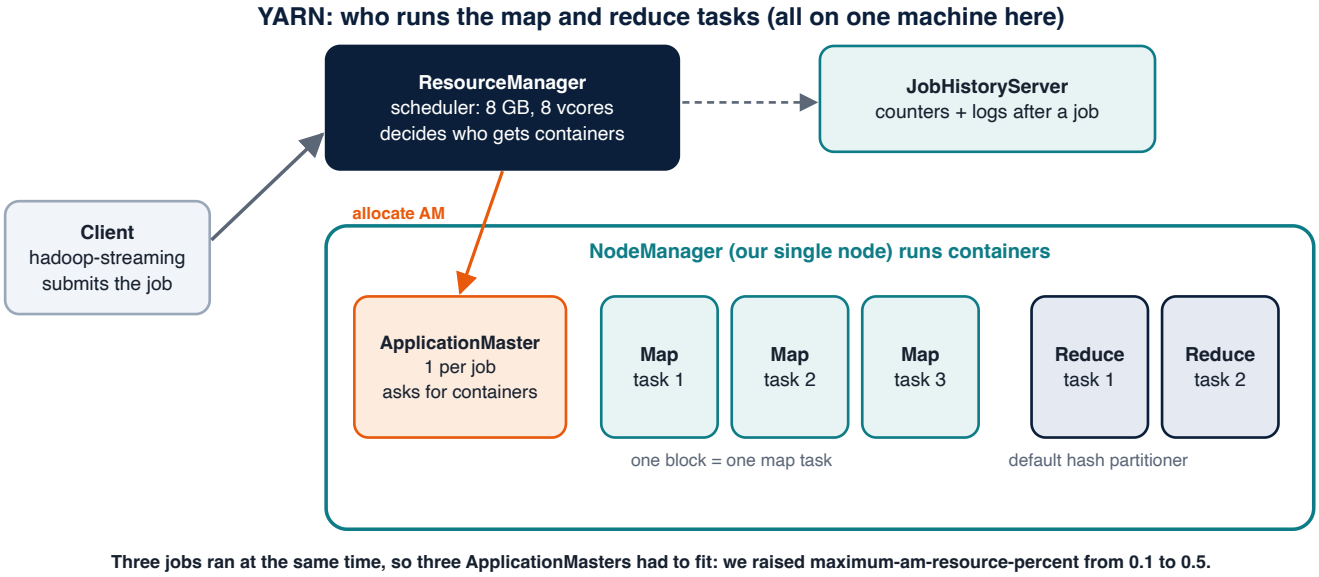


Figure 3. YARN on our single node. Running 3 jobs at once needs 3 ApplicationMasters, which is why the AM resource limit had to be raised.

3.3 MapReduce: the processing model

A job has four steps: **map** turns each input record into (key, value) pairs; the **combiner** (optional) pre-adds pairs on the map side; **shuffle and sort** moves all values of one key to the same reducer; **reduce** combines the values of each key into the output.

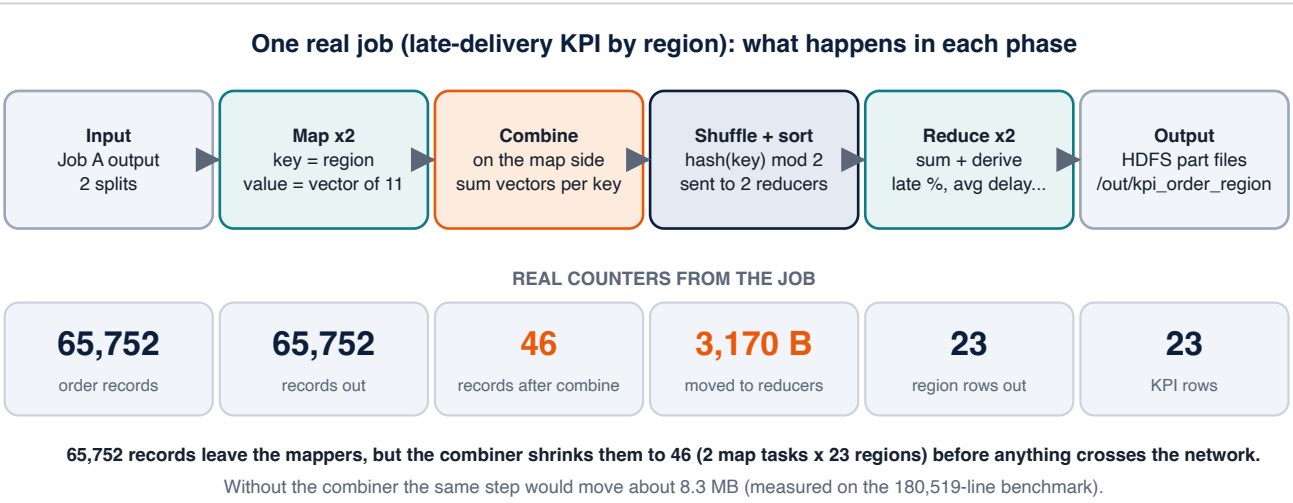


Figure 4. The region KPI job with its real counters. The combiner shrinks 65,752 records to 46 before the shuffle.

> **Keywords to use in the viva:** input split, mapper, key-value pair, partitioner (hash of key mod number of reducers), combiner, shuffle and sort, reducer, output part files, data locality (map task runs where its block is: all 3 of 3 map tasks were data-local), counters, speculative execution, fault tolerance (failed tasks are retried).

3.4 Hadoop Streaming: how we wrote the mappers and reducers in Python

Streaming lets any program that reads **stdin** and writes **stdout** act as the mapper or reducer. This keeps our logic readable, while the Java job (section 5.5) shows the native API. One important difference from Java: a Streaming reducer receives **sorted lines**, not a grouped list, so it must notice when the key changes.

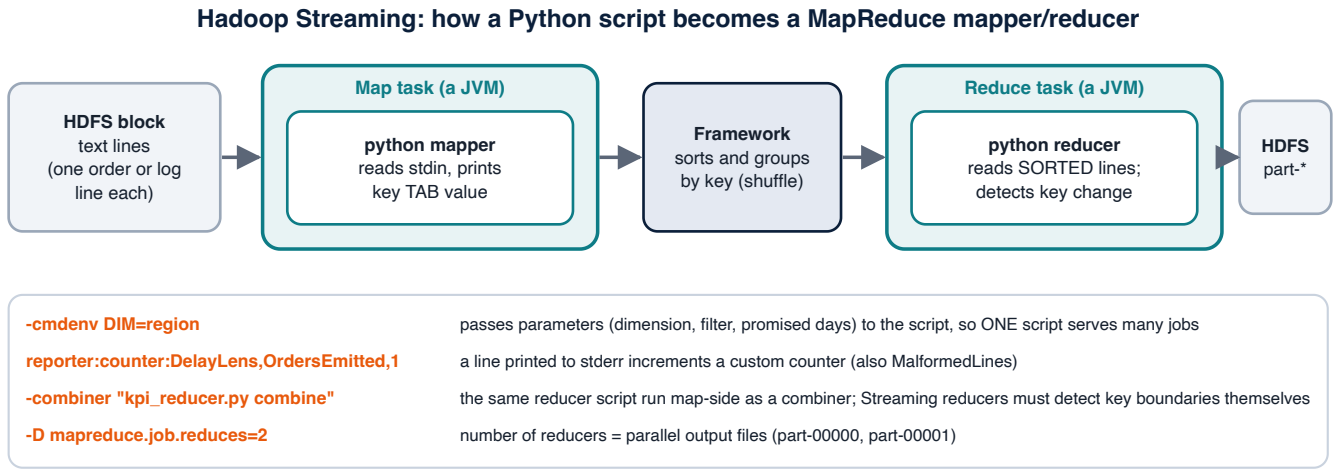


Figure 5. Streaming mechanics. The same generic script is reused for many jobs by passing the dimension with -cmdenv.

What a Streaming job command looks like (this is what our code runs)

```
hadoop jar hadoop-streaming-3.5.0.jar \
  -D mapreduce.job.name=delaylens-kpi-order-region  -D mapreduce.job.reduces=2 \
  -files kpi_mapper.py,kpi_reducer.py \
  -input /delaylens/out/orders  -output /delaylens/out/kpi_order_region \
  -mapper "python3 kpi_mapper.py"  -combiner "python3 kpi_reducer.py combine" \
  -reducer "python3 kpi_reducer.py final" \
  -cmdenv LEVEL=order -cmdenv DIM=region
```

Counters

Counters are numbers Hadoop collects while a job runs. Built-in ones (map input records, combine output records, reduce shuffle bytes, launched map tasks, data-local map tasks) prove what happened. We also add **custom counters** by printing a special line to stderr (`reporter:counter:DelayLens,OrdersEmitted,1`); Job A used it to confirm 65,752 orders and the other jobs count malformed lines.

4. The role of Hadoop in DelayLens

> **One sentence:** Hadoop is the **batch storage and compute engine**: it holds the raw data and produces every KPI number in the project. MongoDB, the agent and the dashboard only *serve* what Hadoop computed.

Hadoop part	What it does for us
HDFS	Holds the cleaned order lines (44.6 MB), the access logs (48.3 MB) and all 33 job outputs under /delaylens/out. Block-based, so each block is processed by its own map task.
MapReduce (Streaming + Java)	It does not clean or predict: it **aggregates**. Job A rolls lines up to orders; KPI jobs group orders or lines by a dimension; click jobs count page views; what-if jobs re-score orders under new promises.
YARN	Schedules the 33 jobs, 3 at a time, and starts the ApplicationMaster and containers for each.
JobHistory + counters	Gives evidence: records in and out, shuffle bytes, tasks launched, time. These went into MongoDB and the dashboard "Big Data pipeline" page.

What Hadoop does not do here (be clear about the boundaries)

- It does **not** answer dashboard clicks: MongoDB does, in milliseconds. A Hadoop job takes 20 to 47 s even on tiny data.
- It does **not** clean the data: pandas does that once on one machine before loading HDFS.
- It does **not** train models: we deliberately have no machine learning.
- It is **not** real-time: it is a batch layer; the pipeline can be re-run daily.

Why Hadoop for this problem?

- The work is **many group-by aggregations over the same records** (region, mode, month, hour, product...). That is exactly the map-key / reduce-sum pattern.
- Map tasks run **per HDFS block in parallel**; the **combiner** shrinks the shuffle (measured 2,600-fold); the design scales by adding nodes.
- Honest limit: at 45 MB, pandas is faster (0.2 s vs 25 s). Hadoop pays a fixed ~20 s start-up per job and earns it back only when data or nodes grow. Our benchmark to 723 MB (64x) shows how the cost grows.

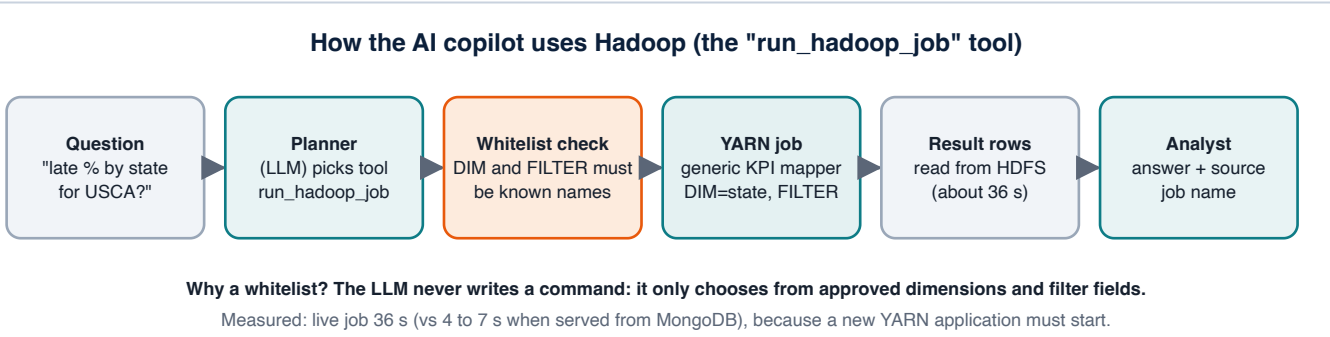


Figure 6. The one place the AI layer calls Hadoop directly: a live MapReduce job, restricted to a whitelist of dimensions and filters.

5. All 33 jobs explained

Jobs are grouped into families. Within a family the mapper and reducer are the same; only the **key** (dimension) changes. Read the family explanations, then use the table at the end as a lookup.

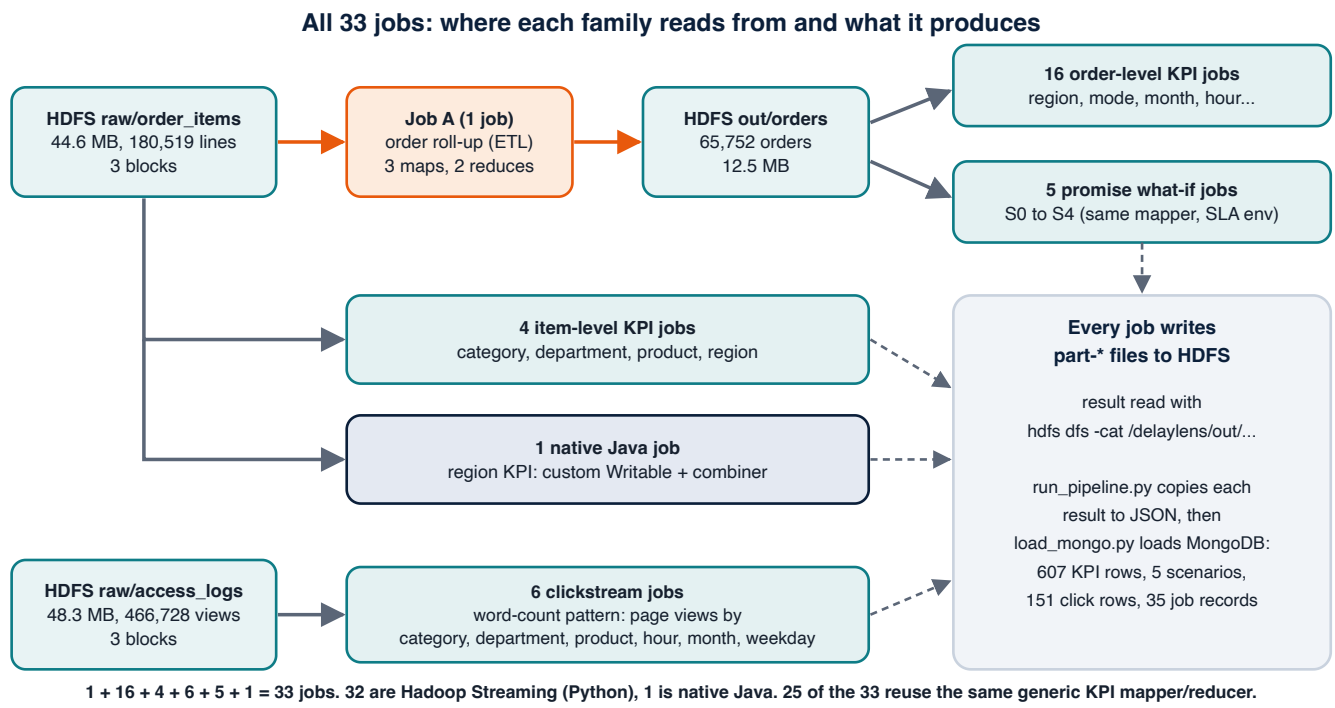


Figure 7. The 33 jobs: 1 ETL + 16 order KPI + 4 item KPI + 6 clickstream + 5 what-if + 1 Java.

5.1 Job A: order roll-up (ETL), 1 job

- **Why:** the DataCo file has one line per order *item*, and delivery facts repeat on every line. Counting lines would over-weight big orders, so all order-level KPIs need one record per order.
- **Map:** read a line, emit **key = order_id**, value = the order attributes + that item's quantity, sales, profit, category, department.
- **Shuffle:** all items of one order meet at one reducer.
- **Reduce:** add item count, quantity, sales, profit; pick the main category (largest sales); emit **one order record** with 26 columns. No combiner (values are not simple sums).
- **Measured:** 180,519 lines in, shuffle 34.5 MB, 65,752 groups, 65,752 records out, 3 map tasks (all data-local), 2 reduce tasks, 29 s.

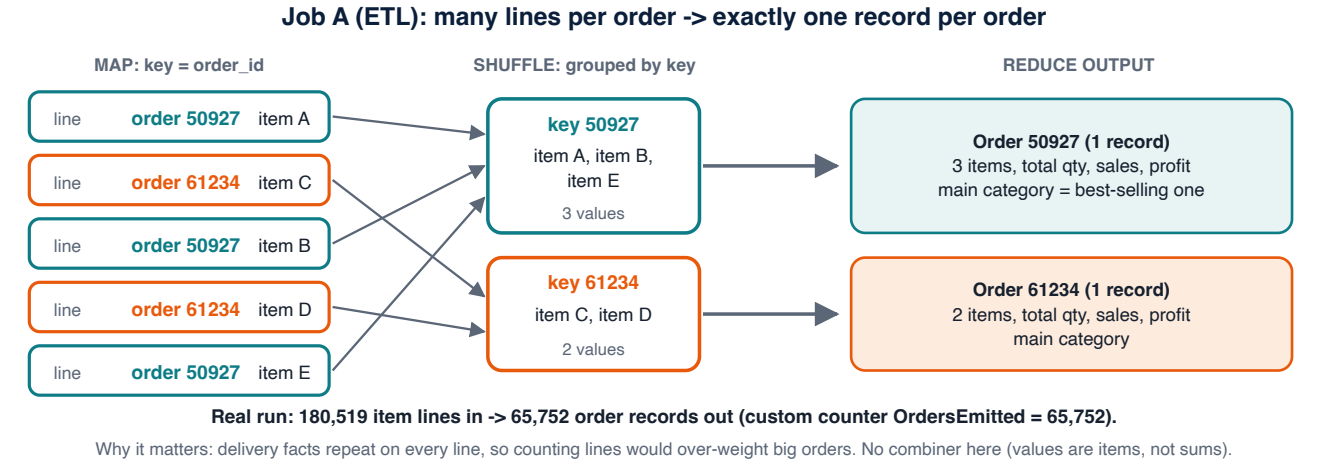


Figure 8. Job A on a toy example, with the real counters underneath.

5.2 The generic KPI job: 16 order-level + 4 item-level + 5 what-if = 25 jobs

One mapper (`kpi_mapper.py`) and one reducer (`kpi_reducer.py`) serve all 25 jobs. The environment variables passed with `-cmdenv` choose what the job does:

Variable	Meaning
LEVEL	**order**: read the Job A output (65,752 orders) or **item**: read the cleaned order lines (180,519).
DIM	The grouping dimension, i.e. what becomes the key (region, market, shipping_mode, month, regionlmode...).
FILTER	Optional equality filter (used when the AI agent launches a job, e.g. market=USCA).
SLA, CUTOFF	Optional what-if: new promised days per shipping mode; Same Day orders placed at or after the cutoff hour are promised next day.

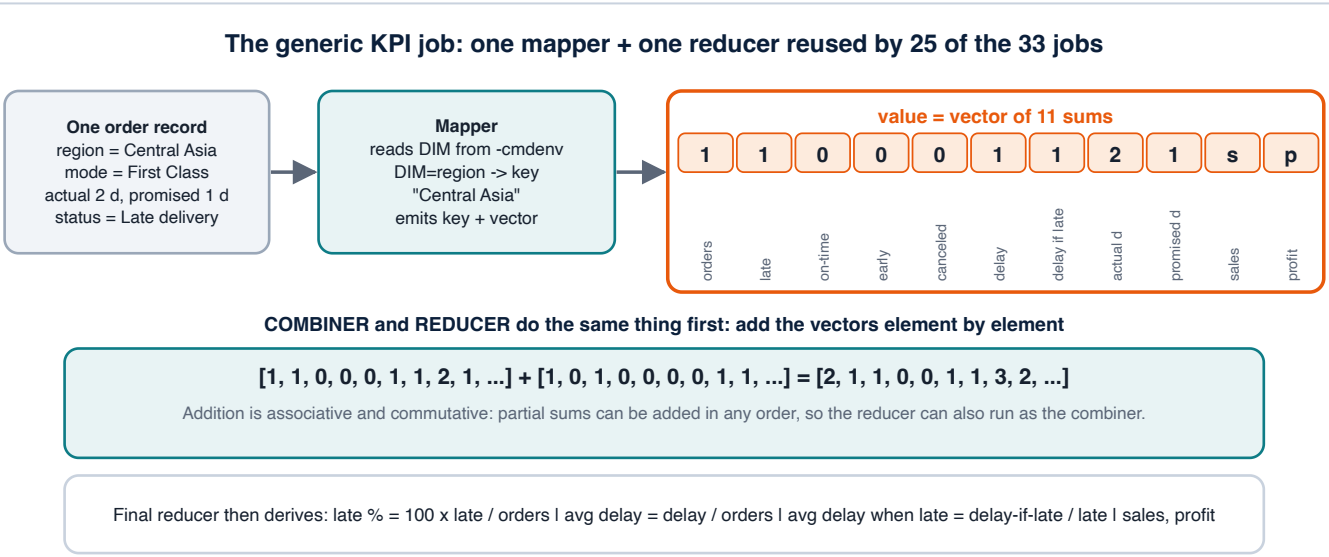


Figure 9. The mapper emits a vector of 11 sums per key. Sums can be added in any order, so the same script works as combiner and reducer.

Classic viva question: why a vector of sums and not percentages? A percentage or average is **not additive**: the average of two averages is wrong when group sizes differ. Counts and sums are additive, so mappers and combiners emit sums, and the **final reducer divides once** (late % = 100 x late / orders). That also makes the combiner legal.

The 16 order-level jobs (input: Job A output)

Group	Dimensions (jobs)	Question it answers
Overall	all	Headline: 65,752 orders, 36,048 late = 54.8%
Where	region (23 groups), market (5), country	Is geography the cause? (49.5% to 57.6% by region)
How shipped	shipping_mode (4), mode_scheduled, scheduled_days, region_mode (92 groups), market_mode	The key finding: mode and promised days drive lateness
When	month (37), dow (weekday), hour	Any trend or Same Day cut-off effect? (flat; after 12:00 always late)
Who / what	segment, payment_type, order_status, delivery_status	Customer segment, payment, open orders, outcome mix

The 4 item-level jobs (input: cleaned order lines)

category, department, product (118) and region at **line** level. Used for product-type questions, because category lives on the item, not the order.

5.3 Clickstream jobs: 6 jobs

- **Input:** 466,728 product page views from the access logs. **Pattern:** the classic **word count**.
- **Map:** emit (dimension value, 1). **Combine and reduce:** add the ones (`sum_reducer.py` is both). **Dimensions:** category, department, product, hour, month, weekday.
- **Measured (product job):** 466,728 records, combiner cut them to 228, reducers wrote 76 products; 1 reducer.
- **Insight:** traffic peaks in the evening (21:00 busiest with 30,273 views), spread evenly across weekdays; web demand does not explain delivery delay.

5.4 Promise what-if jobs: 5 jobs (S0 to S4)

- Same generic mapper, with `SLA` and `CUTOFF` set. Actual shipping days are **never changed**; only the promised days change, and the mapper recomputes **late = actual days > new promised days**.
- The jobs are **cumulative**: each scenario includes the previous changes.

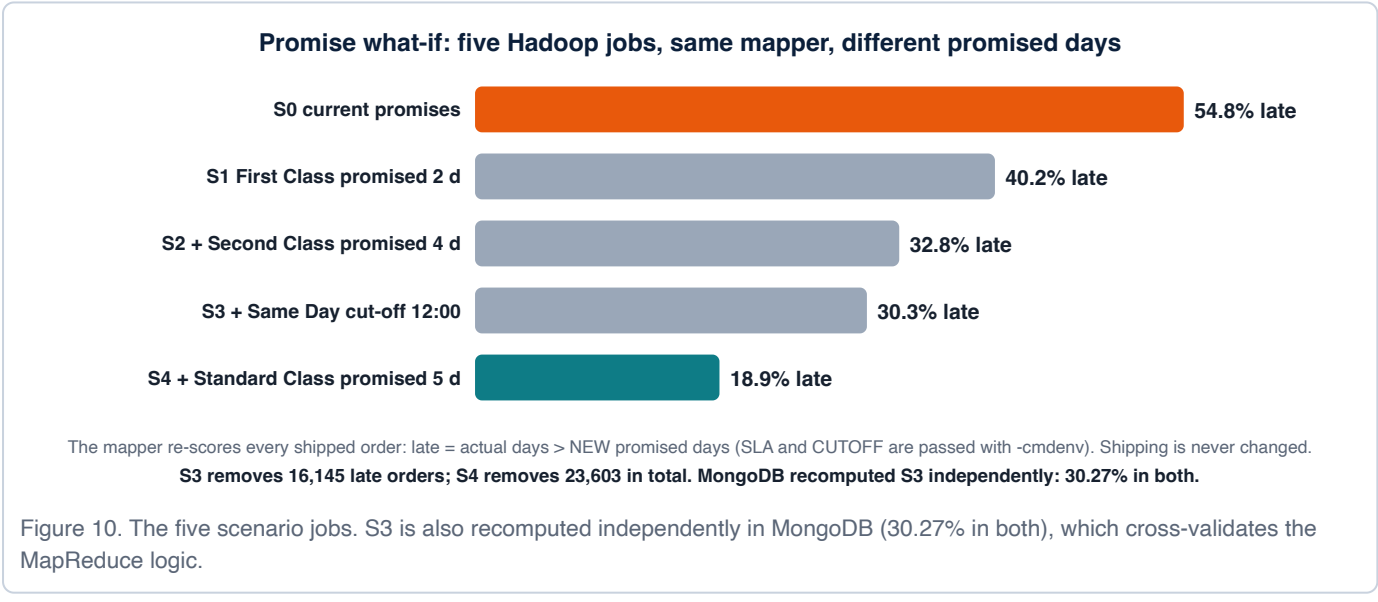


Figure 10. The five scenario jobs. S3 is also recomputed independently in MongoDB (30.27% in both), which cross-validates the MapReduce logic.

Scenario	Change	Late %
S0 current	First 1 d, Second 2 d, Standard 4 d, Same Day 0 d	54.8%
S1	First Class promised 2 days (it always takes 2)	40.2%
S2	+ Second Class promised 4 days (its median)	32.8%
S3	+ Same Day orders after 12:00 promised next day	30.3% (16,145 fewer late orders)
S4	+ Standard Class promised 5 days (commercial trade-off)	18.9% (23,603 fewer in total)

5.5 The native Java job: 1 job

- **Purpose:** show the native MapReduce API and **cross-check** the Streaming result (region KPI at line level: 23 regions, 0 mismatches over 180,519 lines).
- **Parts:** `KpiMapper` (key column set by `-D delaylens.key.col`), `KpiWritable` (our **custom Writable**: count, late, delay sum = 24 bytes), `KpiCombiner`, `KpiReducer`, and a driver run with `ToolRunner`.
- **Result:** 180,519 records, combiner reduced them to 43, shuffle 1,698 bytes (Streaming: 3,202), 21.2 s vs 24.8 s. Java is **14% to 28% faster** because Streaming pipes every record through a separate Python process.
- **Writable** is Hadoop's compact serialization interface (`write` and `readFields`); keys and values must be Writables so they can be sent across the shuffle.

5.6 The agent's ad-hoc job (on demand)

When a question needs a breakdown that was not precomputed, the `run_hadoop_job` tool starts the same generic KPI job with a **whitelisted** DIM and FILTER (Figure 6). Measured: 36 s for "late % by state for the USCA market".

5.7 How long did each job take?

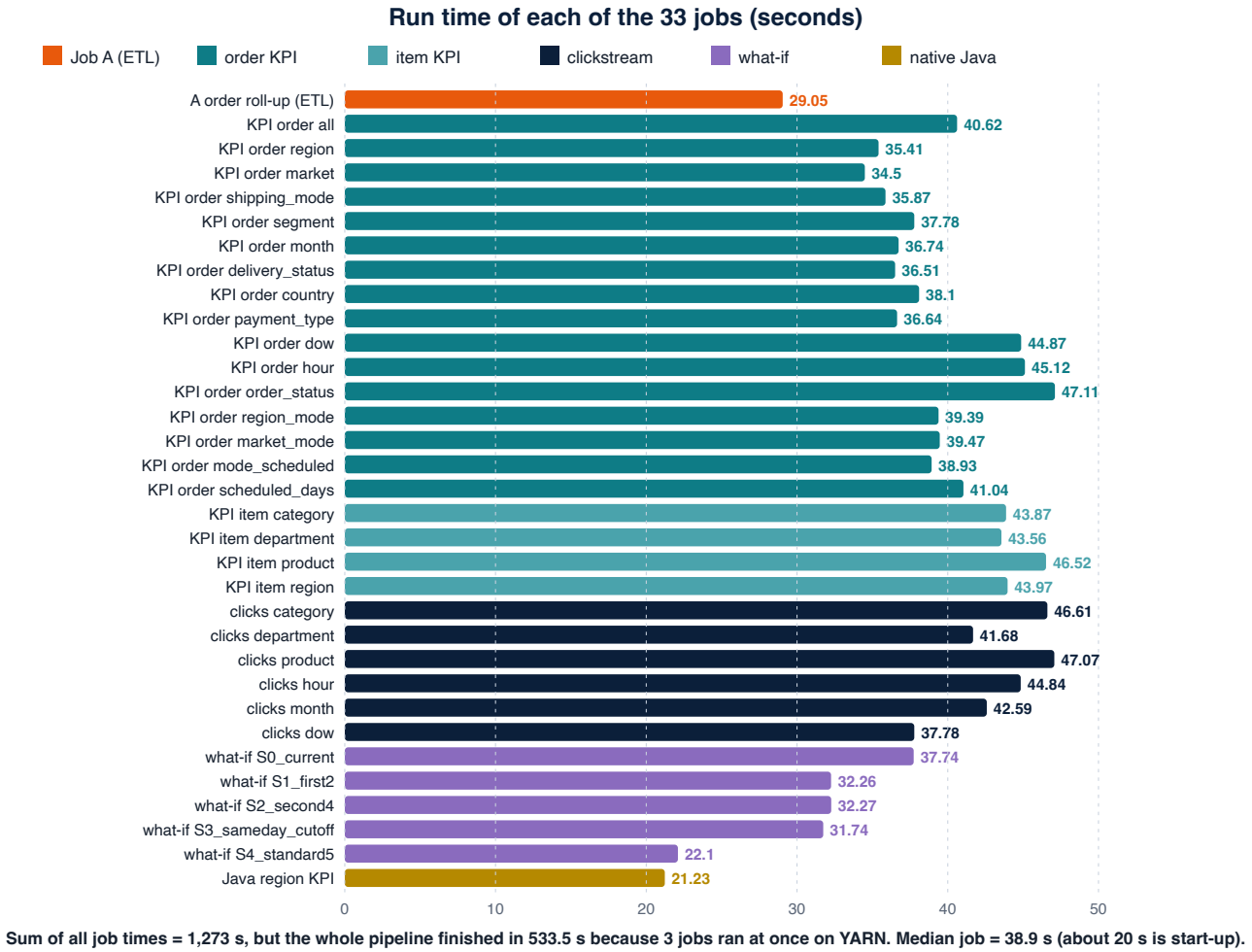


Figure 11. Durations of all 33 jobs. They are all 21 to 47 s: the work is tiny, the fixed YARN/JVM start-up dominates.

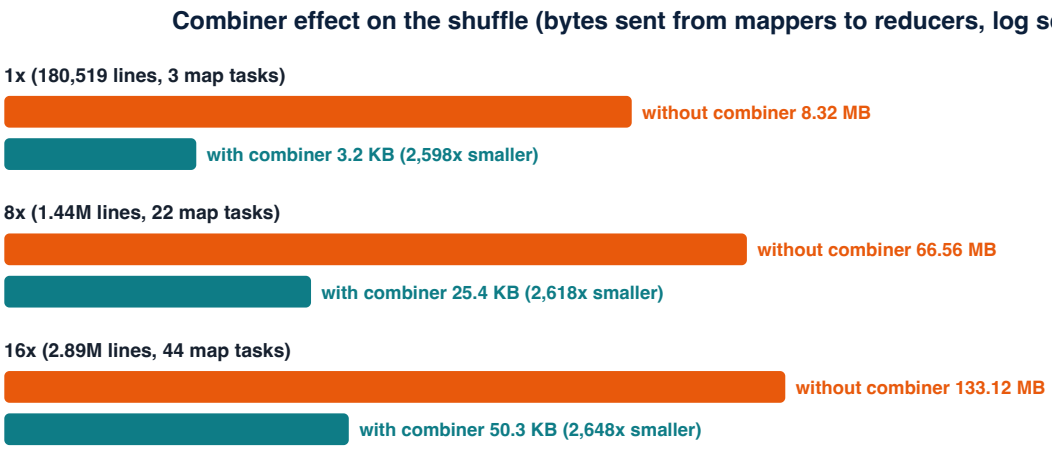
5.8 Lookup table: all 33 jobs with their real counters

Read across a row: records entering the mappers, records left after the combiner (what really crosses the network), rows written, and shuffle size.

#	Job name	Family	Map input records	After combiner	Reduce output rows	Shuffle bytes	Seconds
1	order-rollup	ETL	180,519	0	65,752	34,464,208	29.05
2	kpi-order-all	order KPI	65,752	2	1	188	40.62
3	kpi-order-region	order KPI	65,752	46	23	3,170	35.41
4	kpi-order-market	order KPI	65,752	10	5	756	34.5
5	kpi-order-shipping_mode	order KPI	65,752	8	4	615	35.87
6	kpi-order-segment	order KPI	65,752	6	3	502	37.78
7	kpi-order-month	order KPI	65,752	74	37	4,607	36.74
8	kpi-order-delivery_status	order KPI	65,752	8	4	612	36.51
9	kpi-order-country	order KPI	65,752	315	164	16,652	38.1
10	kpi-order-payment_type	order KPI	65,752	8	4	603	36.64
11	kpi-order-dow	order KPI	65,752	14	7	976	44.87
12	kpi-order-hour	order KPI	65,752	48	24	2,830	45.12
13	kpi-order-order_status	order KPI	65,752	18	9	1,264	47.11
14	kpi-order-region_mode	order KPI	65,752	184	92	13,248	39.39
15	kpi-order-market_mode	order KPI	65,752	40	20	2,967	39.47
16	kpi-order-mode_scheduled	order KPI	65,752	8	4	631	38.93
17	kpi-order-scheduled_days	order KPI	65,752	8	4	533	41.04
18	kpi-item-category	item KPI	180,519	98	50	6,609	43.87
19	kpi-item-department	item KPI	180,519	23	11	1,670	43.56
20	kpi-item-product	item KPI	180,519	226	118	18,732	46.52
21	kpi-item-region	item KPI	180,519	43	23	3,205	43.97
22	clicks-category	clicks	466,728	99	33	2,106	46.61
23	clicks-department	clicks	466,728	18	6	306	41.68
24	clicks-product	clicks	466,728	228	76	9,969	47.07
25	clicks-hour	clicks	466,728	65	24	643	44.84
26	clicks-month	clicks	466,728	14	5	239	42.59
27	clicks-dow	clicks	466,728	21	7	228	37.78
28	sla-S0_current	what-if	65,752	8	4	603	37.74
29	sla-S1_first2	what-if	65,752	8	4	593	32.26
30	sla-S2_second4	what-if	65,752	8	4	592	32.27
31	sla-S3_sameday_cutoff	what-if	65,752	8	4	586	31.74
32	sla-S4_standard5	what-if	65,752	8	4	592	22.1
33	java-kpi-region	Java	180,519	43	23	1,698	21.23

6. Performance and correctness

6.1 The combiner



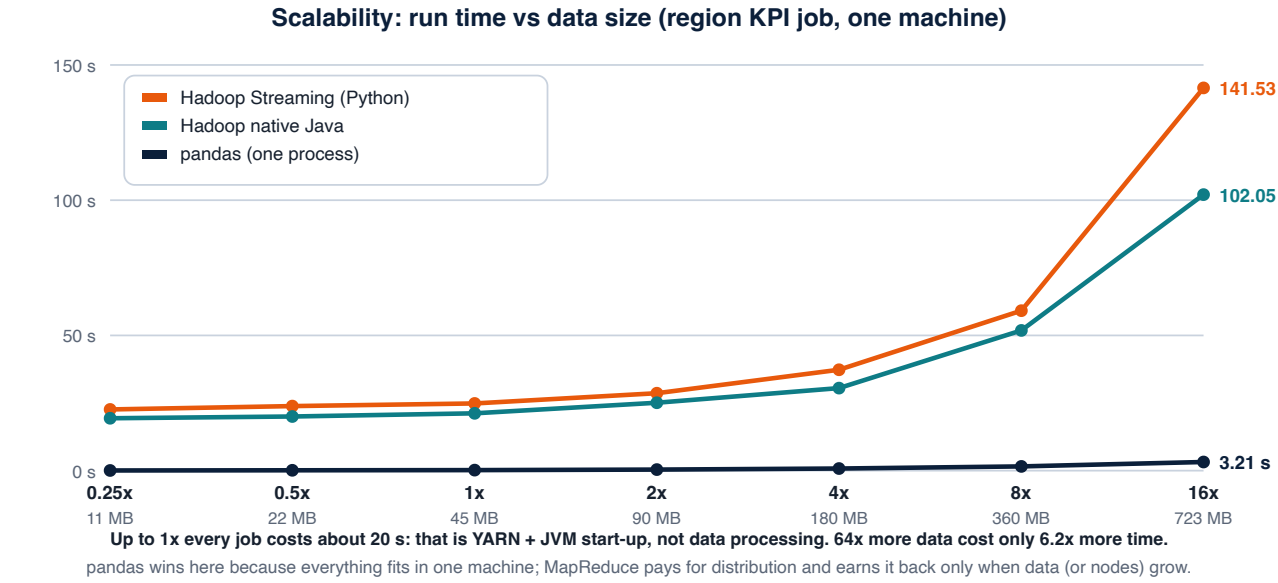
Run time on one machine is NOT always better (24.6 s vs 24.8 s at 1x; 87.9 s vs 59.2 s at 8x; 106 s vs 141 s at 16x): the "network" is a local disk.

On a real multi-node cluster the 2,600-fold smaller shuffle is where the saving would appear.

Figure 12. Shuffle bytes with and without the combiner at three data sizes (measured). The reduction is about 2,600x at every size.

Because the shuffle is already tiny on one machine, run time does not always improve (the combiner also adds a Python pass). The benefit is for multi-node clusters, where shuffle data crosses a real network.

6.2 Scalability benchmark



Up to 1x every job costs about 20 s: that is YARN + JVM start-up, not data processing. 64x more data cost only 6.2x more time.

pandas wins here because everything fits in one machine; MapReduce pays for distribution and earns it back only when data (or nodes) grow.

Figure 13. Region KPI job re-run on 0.25x to 16x data (11 MB to 723 MB, 2 to 44 map tasks).

- **Flat start-up cost:** up to 1x (180,519 lines) every run takes 19 to 25 s: starting the ApplicationMaster and task JVMs, not processing data.
- **Linear growth afterwards:** from 8x to 16x Java took 1.97x as long for 2x the data; Streaming 2.4x. Overall **64x more data cost 6.2x more time** (Streaming 22.7 s to 141.5 s).
- **Waves:** 44 map tasks at 16x run in waves of about 7 containers on one node.
- **pandas is far faster here** (3.2 s at 16x) because the data still fits in one machine's memory: MapReduce earns back its overhead only when data or nodes grow.

6.3 How we know the Hadoop numbers are right

Check	Path 1	Path 2	Result
Orders after roll-up	Hadoop Job A: 65,752 output records	pandas: 65,752 distinct Order Ids	identical
Late orders per region (23)	Hadoop KPI job (Streaming)	MongoDB aggregation on orders	0 mismatches
Line-level region KPI	Hadoop Streaming	native Java MapReduce	0 mismatches (180,519 lines)
Late rate, promise policy S3	Hadoop scenario job	MongoDB simulation	30.27% in both
Late label	Delivery Status = Late delivery	actual days > promised days	100% agreement

7. Cluster setup and problems we solved

Setting	Value	Why
Hadoop / Java	Apache Hadoop 3.5.0, OpenJDK 17	Hadoop 3.5 needs JDK 17 on the server side. Download checksum (SHA-512) verified.
Mode	Pseudo-distributed, 5 daemons	NameNode, DataNode, ResourceManager, NodeManager, JobHistoryServer on one machine (`jps` shows them).
fs.defaultFS	hdfs://localhost:9820	Port 9000 was already taken by a local proxy.
dfs.blocksize	16 MB	So a 45 MB file has 3 blocks and 3 map tasks (visible parallelism).
dfs.replication	1	One DataNode.
YARN resources	8 GB, 8 vcores	Fits the 16 GB laptop and allows several containers.
Start method	`hdfs --daemon start` , `yarn --daemon start`	The usual start scripts need an SSH server; we start the daemons directly.
Reducers	2 (Job A, KPI), 1 (clicks, what-if)	Small outputs; 2 shows the hash partitioner splitting keys.

Problems we hit and how we fixed them (great viva stories)

Problem	Fix and lesson
Daemons would not start (missing log4j and config files)	Copied the default configuration into our own conf folder, which also keeps the project self-contained.
Only one job ran at a time on YARN	The ApplicationMaster limit (`maximum-am-resource-percent`) was 0.1, so only one AM fit in 8 GB. Raised to 0.5 and applied live with `yarn rmadmin -refreshQueues` ; now 3 jobs run concurrently.
Port 9000 conflict	NameNode moved to 9820.
Duplicate-key error loading KPIs into MongoDB	Item-level and order-level region results had the same dimension name; renamed the item-level one so the (dimension, key) index stays unique.
Slow MongoDB `$lookup`	Switched to a local/foreign field join and copied the segment late rate into each order at load time (denormalisation), so the dashboard answers in milliseconds.
Agent crashed on malformed tool arguments	Added a robust argument parser that recovers the first JSON object; the evaluation found this.

8. The rest of the system in brief

The viva is a Big Data course, so Hadoop gets most questions, but you must be able to describe the rest in two or three sentences each.

MongoDB (the serving layer)

- **Why NoSQL:** an order is a natural nested document (customer, shipping, destination, embedded items); KPI outputs have a different shape per dimension; aggregation pipelines and indexes give millisecond answers.
- **Collections:** orders (65,752, items embedded), order_items (180,519), kpis (607 rows), scenarios (5), clickstream (151), hadoop_jobs, agent_logs. Data 244.6 MB, 38.4 MB on disk, 7.3 MB indexes, loaded in 10 s.
- **Indexes:** region + shipping mode, segment late rate, order time, order and shipping status, market; (dimension, key) unique on KPIs.
- **Query times:** 9.9 ms (worst regions) to 140 ms; only the `$lookup` join (Q3) is slow at 1,318 ms, solved by denormalising.

LangGraph agentic copilot

- **Graph:** planner (LLM with function calling, temperature 0) to tools to planner (up to 4 rounds) to analyst (streams the answer). If the LLM gateway is unreachable, a rule-based **fallback** node still answers from the KPIs.
- **9 tools:** overall KPIs, KPI breakdown, order query, at-risk open orders, scenarios, policy simulator, clickstream, pipeline info, live Hadoop job. Inputs are whitelisted.
- **Grounding:** the analyst may only use numbers present in tool results and must name the source job. An early answer wrote an unsupported "20+ point spread"; the prompt was fixed.
- **Evaluation:** 8 test questions with known answers, all correct, 4 to 7 s each (live Hadoop job 36 s); off-topic request politely declined.
- **Model:** azure/deepSeek-V4-Flash through an OpenAI-compatible gateway (reachable only on the company VPN; key in a private .env file, never in code). Served by FastAPI with server-sent events.

Dashboard

Next.js 16, 8 pages: Overview, Regions & markets, Shipping modes, Products & traffic, Promise what-if (precomputed Hadoop scenarios plus a live MongoDB simulator), Open orders at risk, Big Data pipeline, Copilot. Charts: grey for context, one blue accent, tooltips on every mark.

9. Viva question bank with model answers

> Answer in this order: **one-line answer, then our example with a real number**. If you do not know, say what you did measure instead of guessing.

A. Hadoop basics

1. What is Hadoop and what are its main components?

An open-source framework for storing and processing large data on clusters. Three layers: **HDFS** (distributed storage), **YARN** (resource management and scheduling) and **MapReduce** (batch processing model). We used Hadoop 3.5.0.

2. What is HDFS and how does it store a file?

It splits a file into fixed-size blocks, stores them on DataNodes, and the NameNode keeps the metadata. Our 44.6 MB order file became **3 blocks (16 + 16 + 12.6 MB)**.

3. Why did you use 16 MB blocks when the default is 128 MB?

Our file is only 44.6 MB; with 128 MB it would be one block and one map task, so no parallelism to show. 16 MB gives 3 blocks and 3 map tasks. In production we would keep 128 or 256 MB.

4. Why replication factor 1?

We have one DataNode; replication 3 needs three nodes. In production replication 3 gives fault tolerance. On our lab cluster, losing the DataNode would lose data, which is acceptable because the raw files are also on disk.

5. What do the NameNode and DataNode do?

The NameNode is the master: it holds the namespace and the file to block to location map in memory. DataNodes store blocks and send heartbeats and block reports. NameNode failure stops the cluster unless high availability is configured (we did not).

6. What is YARN? Explain ResourceManager, NodeManager, ApplicationMaster and container.

YARN separates resource management from processing. The **ResourceManager** (8 GB, 8 vcores here) allocates resources; the **NodeManager** runs **containers** (CPU + memory slices); each job gets one **ApplicationMaster** that requests containers for its map and reduce tasks.

7. What is pseudo-distributed mode? Is it real Hadoop?

All five daemons (NameNode, DataNode, ResourceManager, NodeManager, JobHistoryServer) run as separate JVMs on one machine. It is real Hadoop, same software and APIs; only the number of nodes is one. The same jobs run unchanged on a multi-node cluster.

8. Why did you not use SSH, and why port 9820?

The standard start scripts use SSH to reach nodes; we start each daemon directly with `hdfs --daemon start` / `yarn --daemon start`. Port 9000 was already used by a local proxy, so the NameNode runs on 9820.

9. Explain MapReduce in one minute.

Map: each input record becomes (key, value). **Shuffle and sort**: Hadoop groups all values of a key onto one reducer. **Reduce**: combines the values of a key into the output. Example: map each order to (region, vector); reduce adds the vectors and computes late %.

10. Walk us through your region KPI job.

Input is the Job A output (65,752 orders, 2 splits). 2 mappers emit (region, 11-value vector): 65,752 records. The combiner adds vectors per region per map task: 46 records. The shuffle moves 3,170 bytes. 2 reducers add again and derive late %, avg delay: 23 region rows to HDFS. (Figure 4)

11. Block versus input split?

A block is a physical piece of a file on HDFS. A split is the logical chunk given to one mapper; by default one split per block. Job A input = 3 blocks = 3 map tasks; the KPI jobs read Job A output, which is two part files = 2 splits.

12. Who decides the number of mappers and reducers?

Mappers follow the number of splits: 3 for 44.6 MB, 22 at 8x and 44 at 16x data. Reducers are set by us: 2 for Job A and KPI jobs, 1 for the small click and what-if outputs.

13. What does the partitioner do?

It decides which reducer gets a key: hash(key) mod number of reducers (default HashPartitioner). All values of one key reach the same reducer. With 2 reducers our region keys are split between part-00000 and part-00001.

14. What is a combiner? Why is it safe in your jobs?

A map-side mini-reducer that pre-aggregates map output to cut shuffle data. Safe because our value is a **vector of sums**: addition is associative and commutative, and its output has the same form as its input. Hadoop may run it zero, one or many times, so the result must not depend on that. Effect: 65,752 to 46 records; shuffle 3.2 KB instead of 8.3 MB.

15. Why can your reducer also act as the combiner?

Because it first just adds vectors. The script has two modes: `combine` prints the summed vector in the same format; `final` adds and then derives the percentages.

16. What is data locality? Did you see it?

The scheduler runs a map task on the node that stores its block, so data is not moved. The counter "Data-local map tasks" was 3 of 3 for Job A. On one node this is trivially true, but it is the real mechanism.

17. What is Hadoop Streaming and why did you use it?

It lets any program that reads stdin and writes stdout be a mapper or reducer. We used Python for readability. Streaming reducers receive **sorted lines** and must detect when the key changes. We also wrote one native Java job to show the real API.

18. What are counters? Which custom ones did you use?

Counters are numbers collected during a job. Built-in: map input records, combine output records, reduce shuffle bytes, launched tasks. Custom (a line printed to stderr): `OrdersEmitted` in Job A (65,752) and `MalformedLines`. They prove what the job did.

19. Why did you need Job A, and why chain jobs?

The file has one line per order item; delivery facts repeat per line. Rolling up by order_id gives order-level facts; otherwise orders with many items would be over-counted. Its output is the input of 21 later jobs (16 KPI + 5 what-if): a chained MapReduce pipeline.

20. What is `-cmdenv` and why use it?

It passes environment variables to the mapper and reducer processes. We use LEVEL, DIM, FILTER, SLA and CUTOFF so **one script serves 25 jobs** instead of writing 25 scripts.

21. Why a vector of sums and not percentages in the mapper?

Percentages and averages are not additive; the average of averages is wrong for unequal group sizes. Sums and counts are additive, so we add them everywhere and divide once in the final reducer.

22. How does the what-if job work?

Same mapper, with SLA (new promised days per mode) and CUTOFF (Same Day hour) set. It recomputes late = actual days > new promised days for every shipped order. Actual shipping never changes. Five cumulative scenarios S0 to S4.

23. Why did you write a Java job as well?

To show the native API (Mapper, Reducer, custom Writable, combiner, ToolRunner) and to cross-check the Streaming result: 0 mismatches over 180,519 lines. Java is also 14% to 28% faster, since Streaming pipes records through a Python process.

24. What is a Writable and why custom?

Hadoop's serialization interface (`write` and `readFields`). Values crossing the shuffle must be Writable. Our `KpiWritable` holds count, late and delay sum (24 bytes), more compact than Java serialization.

25. Why 33 separate jobs instead of one job for all KPIs?

Honest answer: each job re-reads the input, so one job with dimension-prefixed keys (or multiple outputs) would save passes. We chose separate jobs for clarity, independent outputs, easy re-run of one KPI, and because the agent can launch exactly one dimension on demand. Cost: about 20 s start-up each; 3 run concurrently, so the pipeline took 533.5 s instead of 1,273 s.

26. Why does each job take 20 to 47 s for tiny data?

Fixed overhead: YARN allocates an ApplicationMaster, containers, and starts JVMs for every job. Our benchmark shows a flat 19 to 25 s up to 1x data. MapReduce is built for throughput on big batches, not low latency.

27. Why is pandas faster than Hadoop in your benchmark?

Everything fits in one machine's memory, with no scheduling, no disk spill and no shuffle: 3.2 s at 16x versus 141.5 s (Streaming). Hadoop pays for distribution and wins when data exceeds one machine or many nodes share the work.

28. When is Hadoop the right choice?

When data is too big for one machine, the work is batch, and you can add cheap nodes. Not for small data, low latency or iterative algorithms (Spark fits those better).

29. Why not Spark or Hive?

Spark is faster for iterative or in-memory work; Hive/Pig express the same group-bys in SQL or Pig Latin. The course platform is Hadoop MapReduce, and writing mappers and reducers shows partition, shuffle and combine directly. For this workload the dataset suits MapReduce.

30. How does Hadoop handle failures?

A failed task is retried (by default up to 4 attempts) on another container; slow tasks can be re-run speculatively; HDFS blocks are replicated (3 in production); the NameNode can be made highly available. On our lab cluster only task retry applies (replication 1, no HA).

31. What is the shuffle and why is it expensive?

Copying map outputs to the reducers and merge-sorting them by key. It uses network and disk. The combiner exists to shrink it: ours moved 3.2 KB instead of 8.3 MB.

32. What does "Reduce input groups = 65,752" mean?

The reducers saw 65,752 distinct keys, i.e. 65,752 different order_ids, which equals the number of orders.

33. Where are job outputs, and in what format?

Under /delaylens/out/<job>/ as part-* files (tab-separated, sorted by key within each reducer) plus a _SUCCESS marker. Read with `hdfs dfs -cat /delaylens/out/kpi_order_shipping_mode/part-*`.

34. How do results get from HDFS to MongoDB?

run_pipeline.py reads each output with `hdfs dfs -cat`, saves JSON, and load_mongo.py inserts them: 607 KPI rows, 5 scenarios, 151 click rows and 35 job records.

35. What is the JobHistoryServer?

It stores the history and counters of finished jobs (UI on port 19888) so we can inspect them after the job ends; we used it for the counters shown in the report.

36. What went wrong with YARN concurrency?

Only one job ran at a time: the ApplicationMaster resource limit was 0.1, so just one AM fitted in 8 GB. We raised it to 0.5 and applied it live with `yarn rmadmin -refreshQueues`; now 3 jobs run concurrently.

37. Why tab-separated files, not CSV or JSON?

Streaming is line-oriented and splits on a delimiter; product names contain commas and quotes, which would break CSV parsing. Header-less TSV also keeps splits clean.

38. How does a Streaming reducer get its input?

Hadoop sorts map output by key before the reduce phase, so each reducer receives lines grouped by key. The script keeps the current key and flushes its total when the key changes.

39. How does the AI agent use Hadoop?

Through the run_hadoop_job tool: it submits the generic KPI job with a whitelisted DIM and FILTER, waits (about 36 s), reads the result from HDFS and answers. The LLM never writes a shell command.

40. What would you change on a real cluster?

Replication 3, 128 or 256 MB blocks, more reducers, compressed intermediate output, NameNode high availability, YARN queues, a scheduler (e.g. Airflow) to run the pipeline daily, and Spark for iterative work.

41. Hadoop vs MongoDB: why both?

Hadoop is a batch engine for heavy aggregation over files; MongoDB is a low-latency store for serving. Hadoop computes the KPIs once; MongoDB answers dashboard and agent questions in milliseconds.

42. How many daemons did you run and what are they?

Five (`jps`): NameNode, DataNode, ResourceManager, NodeManager, JobHistoryServer.

43. How would you add a new KPI, e.g. orders per payment type?

No new code: run the same generic job with DIM=payment_type (already one of our 16 order-level jobs). For a new dimension: add it to the mapper's dimension table and run the job; the reducer is unchanged.

44. What is speculative execution?

If a task runs much slower than its siblings, Hadoop starts a duplicate on another node and uses whichever finishes first. It protects against slow nodes (not relevant on one node).

45. What is data skew and does it affect you?

When one key has far more records than others, one reducer gets overloaded. Our keys are few (4 modes, 23 regions) but the combiner pre-aggregates per map task, so no single reducer receives a flood of raw records.

B. Data and method

46. Describe the dataset.

DataCo Smart Supply Chain (Mendeley Data): 180,519 order lines with 53 fields (34 kept) for 65,752 orders from Jan 2015 to Jan 2018, plus 469,977 web access-log lines (466,728 product page views after cleaning).

47. How is "late" defined and did you verify it?

An order is late when actual shipping days exceed promised days. We checked it against DataCo's own Delivery Status column: 100% agreement on shipped orders. Canceled orders (4.3%) are not counted as late.

48. What cleaning did you do?

Kept 34 of 53 columns, trimmed whitespace, translated Spanish country and state names, ISO timestamps, derived delay_days, is_late, is_canceled and order month/weekday/hour, removed 3,249 duplicate log lines, exported header-less TSV.

49. Why is the unit of analysis the order, not the line?

Delivery facts belong to the order and repeat per line; counting lines would give multi-item orders more weight. Job A creates the order-level view; product-level questions use the item-level jobs.

50. Is it a data-quality or a business-rule problem?

A business-rule problem: First Class promises 1 day but always takes 2; Same Day orders after 12:00 are always late. The data itself is consistent.

C. MongoDB, agent and results

51. Why MongoDB?

Orders are nested documents; KPI outputs differ in shape per dimension; aggregation pipelines and indexes answer ad-hoc questions in milliseconds. A relational design would need several joins for the same view.

52. Your `$lookup` query was slow. What did you do?

Q3 took 1,318 ms because it joins every open order to the KPI table. We copied the segment late rate into each order at load time (denormalisation, indexed); the dashboard now answers in milliseconds. Classic NoSQL trade-off: duplicate data to speed reads.

53. What makes the copilot "agentic"?

The LLM plans which tools to call, sees the results, may call more (up to 4 rounds), and can launch a new MapReduce job. LangGraph makes this loop an explicit, inspectable graph (planner, tools, analyst, fallback).

54. How do you stop hallucinated numbers?

Tools return the data; the analyst prompt allows only numbers that appear in tool results; arguments are whitelisted; answers name their source job; an evaluation set checks the figures (8 of 8).

55. What if the LLM gateway is down?

The planner routes to a rule-based fallback: a keyword router that still calls the right tool and renders a table from the Hadoop KPIs (answered in 1.6 s and marked offline).

56. What are your main findings?

Shipping mode drives lateness (95.3%, 76.7%, 46.2%, 38.1%); regions differ little (49.5% to 57.6%); late rate is flat over 37 months; fixing promises gives 30.3% (or 18.9%); 11,674 open orders ($6.58M) are at risk.

57. What do you recommend?

Promise First Class in 2 days and Second Class in 4; use a 12:00 cut-off for Same Day; consider Standard 5 days as a commercial trade-off; contact the at-risk open orders first; re-run the pipeline daily.

D. Tough and honest questions

58. 45 MB is not Big Data. Why Hadoop?

Agreed, the raw data is small. The project demonstrates the **architecture and the method**, and we tested scale by growing the data 64x to 723 MB (2.9M rows, 44 map tasks), where time grew only 6.2x. In our data, **variety** (orders, logs, nested documents) and the need for many aggregations are the Big Data features; on one machine pandas would also work, and we say so in the report.

59. Why no machine learning?

The analysis shows lateness follows fixed promise rules per shipping mode (a First Class order always takes 2 days). A what-if simulation explains and fixes this more directly than a classifier, and the case study scope is Hadoop KPIs plus an agentic chatbot.

60. What are the limitations?

One dataset and a single-node cluster; replication 1 and no NameNode high availability; batch only (no streaming); Streaming Python is slower than Java; the combiner gave no speed-up on one machine; the LLM answers depend on the gateway (VPN) and are checked only on 8 test questions.

61. Show us how you would change a job live.

Run the same generic job with a different dimension, e.g. DIM=payment_type, then `hdfs dfs -cat /delaylens/out/kpi_order_payment_type/part-*`. Or ask the copilot: "Run a live Hadoop job: late % by state for the USCA market".

62. What happens if a mapper fails halfway?

The ApplicationMaster reruns that task on another container (up to 4 attempts); the job fails only if a task fails every attempt. Output of failed attempts is discarded, and only the successful attempt is committed.

63. Is the cluster secure?

No: it is a local single-user lab setup without Kerberos or encryption; the LLM key is kept in a private .env file that is never committed. A production cluster would add Kerberos, HDFS permissions and encryption.

64. Who did what in the team?

Agree this together before the viva. Both of you must be able to explain Job A, the generic KPI job, the combiner and the benchmark, and one of you the agent and the dashboard.

10. Live demo script (3 minutes)

Make sure Docker (MongoDB) and the Hadoop daemons are running. If the machine was restarted: `./hadoop/start-hadoop.sh`, start the `delaylens-mongo` container, then `./scripts/start_app.sh`. The copilot needs the company VPN; without it, the fallback still answers.

```
source hadoop/env.sh && jps                                  # 5 Hadoop daemons
hdfs dfs -ls -R /delaylens | head                            # data and job outputs on HDFS
hdfs fsck /delaylens/raw -files -blocks | head               # 16 MB blocks, 3 blocks per file
hdfs dfs -cat /delaylens/out/kpi_order_shipping_mode/part-*  # a MapReduce result (4 rows)
docker exec -i delaylens-mongo mongosh delaylens --quiet < mongo/queries.js
cd agent && ../.venv/bin/python graph.py "Run a live Hadoop job: late % by state for the USCA market"
```

What to open	What to say
NameNode UI, localhost:9870	"Two files, 3 blocks each, one DataNode, replication 1."
YARN UI, localhost:8088	"Three jobs run at once; each has an ApplicationMaster and containers. Total 33 jobs."
JobHistory, localhost:19888	"Counters of a finished job: map input records, combine output records, shuffle bytes."
Dashboard, localhost:3000, Big Data pipeline page	"Job table with counters, benchmark chart and MongoDB query times."
Dashboard, Copilot	Ask: "Which regions have the worst late-delivery rate, and is geography really the cause?" It uses 6 tools and answers: shipping mode.

> **If something fails live:** say what it should show (use the figures in this guide), then show the saved results in `docs/results/` and the screenshots in `docs/screenshots/`. Never improvise numbers.

11. Numbers cheat-sheet

Data	
Order lines / fields	180,519 / 53 (34 kept)
Orders	65,752
Log lines / page views	469,977 / 466,728
Period	Jan 2015 to Jan 2018 (37 months)
File sizes	44.6 MB and 48.3 MB
Duplicates removed	3,249 log lines
Late orders	36,048 = 54.8%
On time or early / canceled	40.8% / 4.3%
Avg delay when late	1.62 days

Hadoop	
Version, Java	3.5.0, OpenJDK 17
Daemons	5
Block size, replication	16 MB, 1
Blocks per file	3
YARN	8 GB, 8 vcores, 3 jobs at once
Jobs	33 (1+16+4+6+5+1)
Streaming / Java	32 / 1
Generic KPI jobs	25 of 33
Pipeline wall time	533.5 s (sum of jobs 1,273 s)
Job time range / median	21 to 47 s / 38.9 s
HDFS put	3.2 s for 93 MB

Job counters	
Job A	180,519 to 65,752; 3 maps, 2 reduces; 29 s
Region KPI	65,752 to 46 (combiner) to 23; shuffle 3,170 B
Product clicks	466,728 to 228 to 76
Java region KPI	180,519 to 43 to 23; 21.2 s
Combiner	8.32 MB to 3.2 KB (2,598x) at 1x
Scale	64x data = 6.2x time (22.7 s to 141.5 s)
Java vs Streaming	14% to 28% faster

Findings	
First / Second Class	95.3% / 76.7% late
Same Day / Standard	46.2% / 38.1% late
Regions / markets	49.5 to 57.6% / 54.1 to 55.3%
What-if S1, S2, S3, S4	40.2, 32.8, 30.3, 18.9%
Late orders removed	16,145 (S3), 23,603 (S4)
Open orders / at risk	33,932 / 11,674 ($6.58M)
MongoDB	607 KPI rows; 244.6 MB; Q3 1,318 ms
Copilot	8 of 8 correct; 4 to 7 s; live job 36 s

Ten one-line answers

1. Hadoop = HDFS + YARN + MapReduce.
2. Block 16 MB so 44.6 MB gives 3 blocks and 3 map tasks.
3. Combiner = map-side mini-reducer; legal because sums are associative and commutative.
4. Job A rolls 180,519 lines to 65,752 orders so order KPIs are not over-weighted.
5. One generic mapper/reducer, parameterised by -cmdenv, runs 25 jobs.
6. Vector of sums, divide once at the end: averages are not additive.
7. What-if: same mapper, new promised days, shipping unchanged.
8. Every job costs about 20 s of YARN start-up, so tiny data does not favour Hadoop.
9. Hadoop computes once, MongoDB serves fast, the agent explains.
10. Cause of delay: shipping-mode promises, not geography.